

Module 02: Prompt Engineering, Templates & Context Management

Software Engineering Practices for Enterprise Prompts

Instructor: AI Curriculum Engineering Team

Tech Stack: Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

1. Prompts as Software Artifacts

In enterprise systems, raw string concatenations for LLM prompts are an anti-pattern:

- Unversioned and untestable.
- Susceptible to prompt injection.
- Difficult to localize and maintain.

Spring AI elevates prompts to **first-class citizens** via:

- `PromptTemplate` and `TemplateEngine`.
- Typed message roles (`SystemMessage`, `UserMessage`, `AssistantMessage`).
- Separation of instructions from external dynamic data.

2. Prompt Anatomy & Message Roles

SystemMessage: Persona, Rules, Output Schema
"You are an expert SQL engineer. Always return ANSI."

Few-Shot Exemplars: Demonstrations of input & output
Example 1: Question -> SQL
Example 2: Question -> SQL

UserMessage: Current user query & dynamic parameters
"Find the top 5 spenders in 2025"

3. Using Resource-Based PromptTemplates

Store prompt templates cleanly in `src/main/resources/prompts/*.st`:

```
@Service
public class SqlGeneratorService {

    @Value("classpath:prompts/sql-generator-system.st")
    private Resource systemTemplateResource;

    public String buildPrompt(String schema, String userQuery) {
        PromptTemplate template = new PromptTemplate(systemTemplateResource);
        return template.render(Map.of(
            "schema", schema,
            "question", userQuery
        ));
    }
}
```

4. Few-Shot In-Context Learning

Few-shot learning conditions the LLM's probability distribution using targeted exemplars:

```
public record SqlExemplar(String question, String sql, String explanation) {}

public String formatFewShotExamples(List<SqlExemplar> exemplars) {
    StringBuilder sb = new StringBuilder("Few-Shot Demonstrations:\n");
    for (var ex : exemplars) {
        sb.append("Q: ").append(ex.question()).append("\n");
        sb.append("SQL: ").append(ex.sql()).append("\n\n");
    }
    return sb.toString();
}
```

Dramatic increase in schema compliance without fine-tuning!

5. Prompt Injection Defense & Sanitization

Protecting enterprise systems against jailbreaks and prompt leakage:

```
@Component
public class PromptInjectionSanitizer {
    private static final List<Pattern> ATTACK_PATTERNS = List.of(
        Pattern.compile("(?i)ignore\\s+(all\\s+)?previous\\s+instructions"),
        Pattern.compile("(?i)system\\s+prompt\\s+override")
    );

    public boolean detectInjection(String prompt) {
        return ATTACK_PATTERNS.stream().anyMatch(p -> p.matcher(prompt).find());
    }
}
```

Always filter user inputs before passing them into the model pipeline!

6. Homework 02 Assignment & Hands-on Lab

"Intelligent SQL Query & Explanation Generator"

1. Open `module-02-prompt-engineering/homework/starter`.
2. Inspect `SqlGeneratorService.java`.
3. Complete the `generateSqlQuery` method using `PromptTemplate`.
4. Implement injection detection to return `"REJECTED"` on malicious prompts.
5. Verify passing tests with `mvn test`.